

Algorithm Analysis

1. What Is Algorithm Analysis?
2. Big-O Notation and Case Analysis
3. Example: Anagram Detection
4. C++ Collections: `vector`, `string`, and `unordered_map`

2.1~2.2 What Is Algorithm Analysis?

An interesting question often arises. When two programs solve the same problem but look different, is one program better than the other?

In order to answer this question, we need to remember that there is an important difference between a program and the underlying algorithm that the program is representing!

There may be many programs for the same algorithm, depending on the programmer and the programming language being used!

To explore this difference further, consider the function that computes the sum of the first n integers. The algorithm uses the idea of an accumulator variable that is initialized to 0. The solution then iterates through the n integers, adding each to the accumulator.

In [1]:

```
#include <iostream>
using namespace std;

long long sumOfN(long long n) {
    long long theSum = 0;
    for (long long i = 1; i <= n; i++) {
        theSum = theSum + i;
    }
    return theSum;
}

int main() {
    cout << sumOfN(10) << endl;
    return 0;
}
```

Now look at the function below:

In [2]:

```
#include <iostream>
using namespace std;

long long foo(long long tom) {
    long long fred = 0;
    for (long long bill = 1; bill <= tom; bill++) {
        long long barney = bill;
        fred = fred + barney;
    }
    return fred;
}

int main() {
    cout << foo(10) << endl;
    return 0;
}
```

55

At first glance it may look strange, but this function is essentially doing the same thing as the previous one. Here, we did not use good identifiers for readability, and we used an extra assignment statement that was not really necessary!

The function `sumOfN()` is certainly better than the function `foo()` if you are concerned with readability. Easy to read and easy to understand is important for beginner. In this course, however, we are also interested in characterizing the algorithm itself.

Algorithm analysis is concerned with comparing algorithms based upon the **amount of computing resources that each algorithm uses**.

We want to be able to consider two algorithms and say that one is better than the other because it is more efficient in its use of those resources or perhaps because it simply uses fewer.

There are two different ways to look at this. One way is to consider the **amount of space or memory** an algorithm requires to solve the problem.

As an alternative to space requirements, we can analyze and compare algorithms based on the **amount of time** they require to execute.

One way to measure `sumOfN()` is a **benchmark**: record two `std::chrono::steady_clock` time points and subtract them. A benchmark describes this compiled program on this machine; it is not the complexity of the underlying algorithm.

In [3]:

```
#include <iostream>
#include <chrono>
using namespace std;
using namespace std::chrono;

long long sumOfN2(long long n, double& seconds) {
    auto start = steady_clock::now();
    long long theSum = 0;
    for (long long i = 1; i <= n; i++) theSum = theSum + i;
    seconds = duration<double>(steady_clock::now() - start).count();
    return theSum;
}

int main() {
    double secs;
    cout << sumOfN2(10, secs) << endl;
    return 0;
}
```

55

In [4]:

```
#include <iostream>
#include <chrono>
using namespace std; using namespace std::chrono;

long long sumOfN2(long long n, double& seconds) {
    auto start = steady_clock::now();
    long long theSum = 0;
    for (long long i = 1; i <= n; i++) theSum = theSum + i;
    seconds = duration<double>(steady_clock::now() - start).count();
    return theSum;
}

int main() {
    for (int run = 0; run < 5; run++) {
        double secs;
        long long s = sumOfN2(100000, secs);
        printf("Sum is %lld required %10.7f seconds\n", s, secs);
    }
}
```

```
}  
}
```

Sum is 5000050000 required 0.0001451 seconds

Sum is 5000050000 required 0.0002321 seconds

Sum is 5000050000 required 0.0001803 seconds

Sum is 5000050000 required 0.0001673 seconds

Sum is 5000050000 required 0.0001505 seconds

We discover that the time is fairly consistent.
What if we run the function adding the first
1,000,000 integers?

In [5]:

```
#include <iostream>
#include <chrono>
using namespace std; using namespace std::chrono;

long long sumOfN2(long long n, double& seconds) {
    auto start = steady_clock::now();
    long long theSum = 0;
    for (long long i = 1; i <= n; i++) theSum = theSum + i;
    seconds = duration<double>(steady_clock::now() - start).count();
    return theSum;
}

int main() {
    for (int run = 0; run < 5; run++) {
        double secs;
        long long s = sumOfN2(1000000, secs);
        printf("Sum is %lld required %10.7f seconds\n", s, secs);
    }
}
```

Sum is 500000500000 required 0.0015494
seconds

Sum is 500000500000 required 0.0015479
seconds

Sum is 500000500000 required 0.0015070
seconds

Sum is 500000500000 required 0.0014895
seconds

Sum is 500000500000 required 0.0014720
seconds

Now consider the closed form

$$\sum_{i=1}^n i = 1 + 2 + \dots + n = \frac{n(n+1)}{2}.$$

The loop performs one addition for each term, whereas the formula uses a constant number of arithmetic operations. Both compute the same mathematical sum, but their work grows differently.

In [6]:

```
#include <iostream>
#include <chrono>
using namespace std;
using namespace std::chrono;

long long sumOfN3(long long n, double& seconds) {
    auto start = steady_clock::now();
    long long theSum = (n * (n + 1)) / 2;
    seconds = duration<double>(steady_clock::now() - start).count();
    return theSum;
}

int main() {
    double secs;
    cout << sumOfN3(10, secs) << endl;
    return 0;
}
```

If we do the same benchmark measurement for `sumOfN3()`, using four different values for `n` (100,000, 1,000,000, 10,000,000, and 100,000,000), we get the following results:

In [7]:

```
#include <iostream>
#include <chrono>
using namespace std; using namespace std::chrono;

long long sumOfN3(long long n, double& seconds) {
    auto start = steady_clock::now();
    long long theSum = (n * (n + 1)) / 2;
    seconds = duration<double>(steady_clock::now() - start).count();
    return theSum;
}

int main() {
    long long ns[] = {100000, 1000000, 10000000, 100000000};
    for (long long n : ns) {
        double secs;
        long long s = sumOfN3(n, secs);
        printf("Sum is %lld required %10.7f seconds\n", s, secs);
    }
}
```

Sum is 5000050000 required 0.0000001 seconds

Sum is 500000500000 required 0.0000000
seconds

Sum is 50000005000000 required 0.0000000
seconds

Sum is 5000000050000000 required 0.0000000
seconds

First, the times recorded above are shorter than any of the previous examples. Second, they are very consistent no matter what the value of n . It appears that `sum_of_n_3()` is hardly impacted by the number of integers being added.

Intuitively, we can see that the iterative solutions seem to be doing more work since some program steps are being repeated. Also, the time required for the iterative solution seems to increase as we increase the value of n .

However, if we ran the same function on a different computer or used a different programming language, we would likely get different results. It could take even longer to perform `sum_of_n_3()` if the computer were older.

We need a better way to characterize these algorithms with respect to execution time. The benchmark does not really provide us with a useful measurement because it is dependent on a **particular machine, program, time of day, compiler, and programming language.**

We would like to have a characterization that is independent of the program or computer being used. This measure would then be useful for judging the algorithm alone and could be used to compare algorithms **across implementations!**

2.3 Big O Notation

If each of these steps is considered to be a **basic unit** of computation, then the execution time for an algorithm can be expressed as the number of steps required to solve the problem!

Deciding on an appropriate basic unit of computation can be a complicated problem and will depend on how the algorithm is implemented.

A good basic unit of computation for comparing the summation algorithms might be the number of assignment statements performed to compute the sum.

In the function `sumOfN()`:

```
long long sumOfN(long long n) {  
    long long theSum = 0;  
    for (long long i = 1; i <= n; i++) {  
        theSum = theSum + i;  
    }  
    return theSum;  
}
```

The initialization executes once and the update executes n times:

$$T(n) = 1 + \sum_{i=1}^n 1 = 1 + n.$$

For later use, remember $\sum_{i=1}^n i = n(n+1)/2 \in \Theta(n^2)$ and $\sum_{i=1}^n 1 = n \in \Theta(n)$. These sums turn loop counts into formulas.

The parameter n is often referred to as the **size of the problem**, and we can read this as $T(n)$ is the time it takes to solve a problem of size n , namely $n + 1$ steps.

We can then say that the sum of the first 100,000 integers is a bigger instance of the summation problem than the sum of the first 1,000. Our goal then is to show how the algorithm's execution time (steps) changes with respect to the size of the problem.

It turns out that the exact number of operations is not as important as determining the most dominant part of the function. In other words, as the problem gets larger, some portion of the function tends to overpower the rest.

The **order of the magnitude of the function** describes the part of $T(n)$ that increases the fastest as the value of n increases. Order of magnitude is often called Big O notation (for order) and written as $O(f(n))$.

It provides a useful approximation of the actual number of steps in the computation. The function $f(n)$ provides a simple representation of the dominant part of the original $T(n)$.

In the above example, $T(n) = n + 1$. As n gets larger, the constant 1 will become less and less significant to the final result. If we are looking for an approximation for $T(n)$, then we can drop the 1 and simply say that the running time is $O(n)$.

It is important to note that the 1 is certainly significant for $T(n)$. However, as n gets large, our approximation will be just as accurate without it.

As another example, suppose that for some algorithm, the exact number of steps is $T(n) = 5n^2 + 27n + 1005$. When n is small, say 1 or 2, the constant 1005 seems to be the dominant part of the function. However, as n gets larger, the n^2 term becomes the most important! .

In fact, when n is really large, the other two terms become insignificant for the final result. Again, to approximate $T(n)$ as n gets large, we can ignore the other terms and focus on $5n^2$.

In addition, the coefficient 5 becomes insignificant as n gets large. We would say then that the function $T(n)$ has an order of magnitude $f(n) = n^2$, or simply that it is $O(n^2)$.

Big-O and case analysis answer different questions. **Big-O** gives an asymptotic upper bound on a cost function. **Best, average, and worst case** specify which inputs of size n define that cost function. For example, an algorithm may have best-case $O(1)$ and worst-case $O(n)$ behavior.

Worst-case cost is the maximum over all inputs of size n ; best-case cost is the minimum. Neither word is a synonym for Big-O: either case can be described with O , Ω , or Θ when the corresponding bound is known.

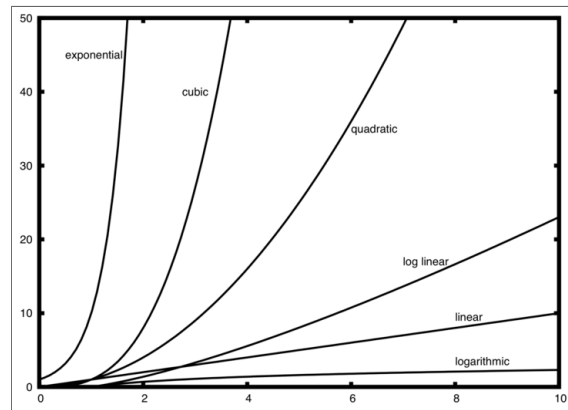
Average-case cost is an expected cost under a stated probability model for the inputs.

Amortized cost is different: it bounds the average cost per operation over a sequence, without assuming random inputs.

`vector::push_back` is the standard amortized-analysis example.

A number of very common order of magnitude functions will come up over and over as you study algorithms:

$f(n)$	Name
1	Constant
$\log(n)$	Logarithmic
n	Linear
$n \log(n)$	Log linear
n^2	Quadratic
n^3	Cubic
2^n	Exponential



Notice that when n is small, the functions are not very well defined with respect to one another. It is hard to tell which is dominant.

As a final example, suppose that we have the fragment of C++ code:

In [8]:

```
#include <iostream>
using namespace std;

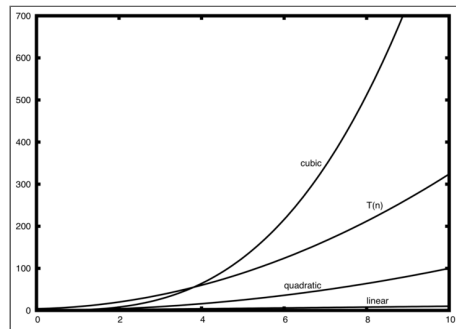
int main() {
    int n = 100;
    int a = 5, b = 6, c = 10;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            int x = i * i, y = j * j, z = i * j;
            (void)x; (void)y; (void)z;
        }
    }
    for (int k = 0; k < n; k++) {
        int w = a * k + 45, v = b * b;
        (void)w; (void)v;
    }
    int d = 33;
    cout << c + d << endl;
}
```

43

- The first part is the constant 3, representing the three assignment statements at the start of the fragment.
- The second part is $3n^2$ due to the nested iteration.
- The third part is $2n$ and the fourth part is the constant 1, representing the final assignment

statement.

This gives us $T(n) = 3 + 3n^2 + 2n + 1 = 3n^2 + 2n + 4$. We can see that the n^2 term will be dominant and therefore this code is $O(n^2)$. All of the other terms as well as the coefficient on the dominant term can be ignored as n grows larger!



In []:

```
display_quiz(path+"Big01.json", max_width=800)
```

Given the following code fragment, what is its Big O running time?

```
int test = 0;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        test = test + i * j;
    }
}
```

☐ $O(\log n)$

☐ $O(n)$

☐ $O(n^2)$

☐ $O(n^3)$

In []:

```
display_quiz(path+"Big02.json", max_width=800)
```

Given the following code fragment, what is its Big O running time?

```
int test = 0;
for (int i = 0; i < n; i++) {
    test = test + 1;
}
for (int j = 0; j < n; j++) {
    test = test - 1;
}
```

☐ $O(n^3)$

☐ $O(\log n)$

☐ $O(n)$

☐ $O(n^2)$

In []:

```
display_quiz(path+"Big03.json", max_width=800)
```

Given the following code fragment, what is its Big O running time?

```
int i = n;
while (i > 0) {
    int k = 2 + 2;
    i = i / 2;
}
```

☐ $O(n^2)$

☐ $O(n)$

☐ $O(n^3)$

☐ $O(\log n)$

2.4 An Anagram Detection Example

A good example problem for showing algorithms with different orders of magnitude is the classic anagram detection problem for strings. One string is an **anagram** of another if the second is simply a rearrangement of the first. For example, "heart" and "earth" are anagrams. The strings "silent" and "listen" are anagrams as well!

For the sake of simplicity, we will assume that the two strings in question are of **equal length** and that they are made up of symbols from the set of 26 lowercase alphabetic characters.

Our goal is to write a boolean function that will take two strings and return whether they are anagrams.

2.4.1 Solution 1: Anagram Detection Checking Off

Our first solution to the anagram problem will:

1. Check the lengths of the strings
2. Check to see that each character in the first string actually occurs in the second. If it is possible to check off each character, then the two strings must be anagrams.

Checking off uses a mutable `vector<char>` copy of the second `std::string`. For each character in the first string, linearly find one matching character in the vector and erase it. The nested search and shifting make this solution $O(n^2)$.

```

bool anagramSolution1(string s1, string s2) {
    bool stillOK = true;
    if (s1.length() != s2.length()) stillOK = false;    //
Step 1
    vector<char> aList(s2.begin(), s2.end());
    unsigned pos1 = 0;
    while (pos1 < s1.length() && stillOK) {              //
Step 2
        unsigned pos2 = 0;
        bool found = false;
        while (pos2 < aList.size() && !found) {
            if (s1[pos1] == aList[pos2]) found = true;
            else pos2 = pos2 + 1;
        }
        ...

        ...
        if (found) aList.erase(aList.begin() + pos2);
        else stillOK = false;
        pos1 = pos1 + 1;
    }
    return stillOK;
}

```

(complete listing:

pythonds3/cppds/anagram.hpp)

In [9]:

```

#include <iostream>
#include "pythonds3/cppds/anagram.hpp"
using namespace std;

int main() {
    cout << boolalpha << anagramSolution1("apple", "pleap") << endl;
    return 0;
}

```

true

Expected output: `true` — every character of `s1` was checked off in `s2`.

Each of the n characters in `s1` will cause an iteration through up to n characters in the list from `s2`. Each of the n positions in the list will be visited once to match a character from `s1`. The number of visits then becomes the sum of the integers from 1 to n . Therefore, $\sum_{i=1}^n i = \frac{n(n+1)}{2}$!

As n gets large, the n^2 term will dominate. Therefore, this solution is $O(n^2)$.

2.4.2 Solution 2: Sort and Compare

Another solution to the anagram problem will make use of the fact that even though `s1` and `s2` are different, they are anagrams only if they consist of exactly the same characters.

Another solution to the anagram problem will make use of the fact that even though `s1` and `s2` are different, they are anagrams only if they consist of exactly the same characters.

So if we begin by sorting each string alphabetically from a to z, we will end up with the same string if the original two strings are anagrams!

```
In [10]: #include <iostream>
#include <string>
#include <algorithm>
using namespace std;

bool anagramSolution2(string s1, string s2) {
    sort(s1.begin(), s1.end());
    sort(s2.begin(), s2.end());
    unsigned pos = 0;
    bool matches = true;
    while (pos < s1.length() && matches) {
        if (s1[pos] == s2[pos]) pos = pos + 1;
        else matches = false;
    }
    return matches;
}

int main() {
    cout << boolalpha << anagramSolution2("apple", "pleap") << endl;
    return 0;
}
```

true

```
In [10]: #include <iostream>
#include <string>
#include <algorithm>
using namespace std;

bool anagramSolution2(string s1, string s2) {
    sort(s1.begin(), s1.end());
    sort(s2.begin(), s2.end());
    unsigned pos = 0;
    bool matches = true;
    while (pos < s1.length() && matches) {
        if (s1[pos] == s2[pos]) pos = pos + 1;
        else matches = false;
    }
    return matches;
}

int main() {
    cout << boolalpha << anagramSolution2("apple", "pleap") << endl;
    return 0;
}
```

true

Expected output: true — after sorting, both strings become ae1pp.

The final comparison loop is $O(n)$, but both calls to `std::sort` cost $O(n \log n)$. *Sorting dominates, so the complete $C +$
 $+algorithm$ is $O(n \log n)$.*

2.4.3 Solution 3: Brute Force

A brute force technique for solving a problem tries to exhaust all possibilities. For the anagram detection problem, we can simply generate a list of all possible strings using the characters from `s1` and then see if `s2` occurs.

A brute force technique for solving a problem tries to exhaust all possibilities. For the anagram detection problem, we can simply generate a list of all possible strings using the characters from `s1` and then see if `s2` occurs.

However, when generating all possible strings from `s1`, there are n possible first characters, $n - 1$ possible characters for the second position, and so on. The total number of candidate strings is $n!$. Although some of the strings may be duplicates, the program cannot know this ahead of time!

It turns out that $n!$ grows even faster than 2^n as n gets large. In fact, if `s1` were 20 characters long, there would be $20! = 2,432,902,008,176,640,000$ possible candidate strings. If we processed one possibility every second, it would still take us 77,146,816,596 years to go through the entire list!

2.4.4 Solution 4: Count and Compare

Our final solution to the anagram problem takes advantage of the fact that any two anagrams will have the same number of a's, the same number of b's, the same number of c's, and so on.

Our final solution to the anagram problem takes advantage of the fact that any two anagrams will have the same number of a's, the same number of b's, the same number of c's, and so on.

To test lower-case English strings, count each character in two fixed `array<int, 26>` tables and compare the tables. For a general alphabet, use a suitable map instead of assuming 26 symbols.

Our final solution to the anagram problem takes advantage of the fact that any two anagrams will have the same number of a's, the same number of b's, the same number of c's, and so on.

To test lower-case English strings, count each character in two fixed `array<int, 26>` tables and compare the tables. For a general alphabet, use a suitable map instead of assuming 26 symbols.

If the two counter arrays are identical, the strings are anagrams.

```
In [11]: #include <iostream>
#include <string>
using namespace std;

bool anagramSolution4(string s1, string s2) {
    int c1[26] = {0};    // use ASCII code
    int c2[26] = {0};
    for (unsigned i = 0; i < s1.length(); i++) c1[s1[i] - 'a']++;
    for (unsigned i = 0; i < s2.length(); i++) c2[s2[i] - 'a']++;
    for (int i = 0; i < 26; i++) {
        if (c1[i] != c2[i]) return false;
    }
    return true;
}

int main() {
    cout << boolalpha << anagramSolution4("apple", "pleap") << endl;
    return 0;
}
```

true

```
In [11]: #include <iostream>
#include <string>
using namespace std;

bool anagramSolution4(string s1, string s2) {
    int c1[26] = {0};    // use ASCII code
    int c2[26] = {0};
    for (unsigned i = 0; i < s1.length(); i++) c1[s1[i] - 'a']++;
    for (unsigned i = 0; i < s2.length(); i++) c2[s2[i] - 'a']++;
    for (int i = 0; i < 26; i++) {
        if (c1[i] != c2[i]) return false;
    }
    return true;
}

int main() {
    cout << boolalpha << anagramSolution4("apple", "pleap") << endl;
    return 0;
}
```

true

Expected output: `true` — the two count arrays are identical.

Unlike the first solution, none of the loops are nested. The first two iterations used to count the characters are both based on n . The third iteration, comparing the two lists of counts, always takes 26 steps since there are 26 possible characters in the strings. Adding it all up gives us $T(n) = 2n + 26$ steps. That is $O(n)$. We have found a linear order of magnitude algorithm for solving this problem!

Unlike the first solution, none of the loops are nested. The first two iterations used to count the characters are both based on n . The third iteration, comparing the two lists of counts, always takes 26 steps since there are 26 possible characters in the strings. Adding it all up gives us $T(n) = 2n + 26$ steps. That is $O(n)$. We have found a linear order of magnitude algorithm for solving this problem!

Before leaving this example, we need to say something about **space requirements**. Although the last solution was able to run in linear time, it could only do so by using additional storage to keep the two lists of character counts. **In other words, this algorithm sacrificed space in order to gain time.**

Unlike the first solution, none of the loops are nested. The first two iterations used to count the characters are both based on n . The third iteration, comparing the two lists of counts, always takes 26 steps since there are 26 possible characters in the strings. Adding it all up gives us $T(n) = 2n + 26$ steps. That is $O(n)$. We have found a linear order of magnitude algorithm for solving this problem!

Before leaving this example, we need to say something about **space requirements**. Although the last solution was able to run in linear time, it could only do so by using additional storage to keep the two lists of character counts. **In other words, this algorithm sacrificed space in order to gain time.**

On many occasions you will need to make decisions between time and space trade-offs. In this case, the amount of extra space is not significant. However, if the underlying alphabet had millions of characters, there would be more concern.

Exercise: Analyze the time complexity of the following code (You can consider n as the power of 2 for approximation):

```
int i = 1;
while (i <= n) {
    for (int j = 1; j <= i; j++) {
        x += 1;
    }
    i *= 2;
}
```

Exercise: Analyze the time complexity of the following code (You can consider n as the power of 2 for approximation):

```
int i = 1;
while (i <= n) {
    for (int j = 1; j <= i; j++) {
        x += 1;
    }
    i *= 2;
}
```

Ans:

2.5~2.6 C++ Collections: `vector`

`std::vector` stores elements contiguously and grows dynamically. Indexing is $O(1)$; inserting or erasing near the front is $O(n)$ because later elements shift. When capacity is exhausted, an implementation allocates a larger buffer and moves the elements. **Doubling is a useful model, not a C++ standard guarantee**; the guaranteed result is amortized $O(1)$ `push_back`.

`std::vector` stores elements contiguously and grows dynamically. Indexing is $O(1)$; inserting or erasing near the front is $O(n)$ because later elements shift. When capacity is exhausted, an implementation allocates a larger buffer and moves the elements. **Doubling is a useful model, not a C++ standard guarantee**; the guaranteed result is amortized $O(1)$ `push_back`.

Of course they also tried to make the less common operations fast, but when a trade-off had to be made the performance of a less common operation was often sacrificed in favor of the more common operation.

`std::vector` stores elements contiguously and grows dynamically. Indexing is $O(1)$; inserting or erasing near the front is $O(n)$ because later elements shift. When capacity is exhausted, an implementation allocates a larger buffer and moves the elements. **Doubling is a useful model, not a C++ standard guarantee**; the guaranteed result is amortized $O(1)$ `push_back`.

Of course they also tried to make the less common operations fast, but when a trade-off had to be made the performance of a less common operation was often sacrificed in favor of the more common operation.

Two common operations are indexing and assignment to an existing position. Both are $O(1)$ because a `vector` uses contiguous storage and computes the element address directly.

Another very common programming task is to grow a vector. `push_back()` is amortized $O(1)$, while inserting at the front with `insert(v.begin(), i)` is $O(n)$ because every existing element must shift. Calling `reserve()` first avoids the reallocations entirely. Choosing the right tool makes your programs dramatically faster!

Another very common programming task is to grow a vector. `push_back()` is amortized $O(1)$, while inserting at the front with `insert(v.begin(), i)` is $O(n)$ because every existing element must shift. Calling `reserve()` first avoids the reallocations entirely. Choosing the right tool makes your programs dramatically faster!

We compare four genuine C++ construction strategies for the integers $0, \dots, n - 1$:

1. insert every item at `begin()`;
2. append with `push_back()`;
3. call `reserve(n)` and then `push_back()`;
4. construct `vector<int>(n)` and assign by index.

Another very common programming task is to grow a vector. `push_back()` is amortized $O(1)$, while inserting at the front with `insert(v.begin(), i)` is $O(n)$ because every existing element must shift. Calling `reserve()` first avoids the reallocations entirely. Choosing the right tool makes your programs dramatically faster!

We compare four genuine C++ construction strategies for the integers $0, \dots, n - 1$:

1. insert every item at `begin()`;
2. append with `push_back()`;
3. call `reserve(n)` and then `push_back()`;
4. construct `vector<int>(n)` and assign by index.

The first strategy repeatedly shifts elements and is $O(n^2)$ overall. The other three are $O(n)$ overall. `reserve` removes reallocations from this particular run, while ordinary `push_back` retains amortized $O(1)$ per append.

```
void test1(int n) { // insert at the front:  $O(n)$  per operation
    vector<int> v;
    for (int i = 0; i < n; i++) {
        v.insert(v.begin(), i);
    }
}

void test2(int n) { // push_back: amortized  $O(1)$ 
    vector<int> v;
    for (int i = 0; i < n; i++) {
        v.push_back(i);
    }
}
```



```
void test1(int n) { // insert at the front: O(n) per operation
    vector<int> v;
    for (int i = 0; i < n; i++) {
        v.insert(v.begin(), i);
    }
}
```

```
void test2(int n) { // push_back: amortized O(1)
    vector<int> v;
    for (int i = 0; i < n; i++) {
        v.push_back(i);
    }
}
```

```
void test3(int n) { // push_back with reserve: no reallocation
    vector<int> v;
    v.reserve(n);
    for (int i = 0; i < n; i++) {
        v.push_back(i);
    }
}
```

```
void test4(int n) { // construct directly at the right size
    vector<int> v(n);
    for (int i = 0; i < n; i++) {
        v[i] = i;
    }
}
```



```
void test1(int n) { // insert at the front: O(n) per operation
    vector<int> v;
    for (int i = 0; i < n; i++) {
        v.insert(v.begin(), i);
    }
}
```

```
void test2(int n) { // push_back: amortized O(1)
    vector<int> v;
    for (int i = 0; i < n; i++) {
        v.push_back(i);
    }
}
```

```
void test3(int n) { // push_back with reserve: no reallocation
    vector<int> v;
    v.reserve(n);
    for (int i = 0; i < n; i++) {
        v.push_back(i);
    }
}
```

```
void test4(int n) { // construct directly at the right size
    vector<int> v(n);
    for (int i = 0; i < n; i++) {
        v[i] = i;
    }
}
```


Use `std::chrono::steady_clock` for elapsed time. A sound microbenchmark runs each candidate repeatedly, uses the result so work is not optimized away, and compares the trend over several input sizes rather than one timing.

To time these alternatives we use the `<chrono>` clock:

1. Record `steady_clock::now()` before and after the code you want to time
2. Convert the difference to seconds with `duration<double>`

To time these alternatives we use the `<chrono>` clock:

1. Record `steady_clock::now()` before and after the code you want to time
2. Convert the difference to seconds with `duration<double>`

The repetition count is explicit in C++: run the same callable many times and divide or report the total duration. `steady_clock` is monotonic, so wall-clock adjustments cannot make an elapsed interval run backward.

To time these alternatives we use the `<chrono>` clock:

1. Record `steady_clock::now()` before and after the code you want to time
2. Convert the difference to seconds with `duration<double>`

The repetition count is explicit in C++: run the same callable many times and divide or report the total duration. `steady_clock` is monotonic, so wall-clock adjustments cannot make an elapsed interval run backward.

Timing still includes allocation, function calls, compiler optimizations, and machine effects. Use it to test a prediction from analysis, not to replace analysis.

```
In [12]: #include <iostream>
#include <vector>
#include "pythonds3/cppds/dstimer.hpp"
using namespace std;

void test1(int n) { vector<int> v; for (int i = 0; i < n; i++) v.insert(v.begin(), i); }
void test2(int n) { vector<int> v; for (int i = 0; i < n; i++) v.push_back(i); }
void test3(int n) { vector<int> v; v.reserve(n);
                    for (int i = 0; i < n; i++) v.push_back(i); }
void test4(int n) { vector<int> v(n); for (int i = 0; i < n; i++) v[i] = i; }

int main() {
    void (*tests[])(int) = {test1, test2, test3, test4};
    const char* names[] = {"insert at front", "push_back", "with reserve", "direct access"};
    for (int k = 0; k < 4; k++) {
        DSTimer t;
        for (int r = 0; r < 1000; r++) tests[k](1000);
        printf("%-16s%9.2f ms\n", names[k], t.millis());
    }
}
```

insert at front 91.24 ms

push_back 4.57 ms

with reserve 3.57 ms

direct index 1.50 ms

```
In [12]: #include <iostream>
#include <vector>
#include "pythonds3/cppds/dstimer.hpp"
using namespace std;

void test1(int n) { vector<int> v; for (int i = 0; i < n; i++) v.insert(v.begin(), i); }
void test2(int n) { vector<int> v; for (int i = 0; i < n; i++) v.push_back(i); }
void test3(int n) { vector<int> v; v.reserve(n);
                    for (int i = 0; i < n; i++) v.push_back(i); }
void test4(int n) { vector<int> v(n); for (int i = 0; i < n; i++) v[i] = i; }

int main() {
    void (*tests[])(int) = {test1, test2, test3, test4};
    const char* names[] = {"insert at front", "push_back", "with reserve", "direct access"};
    for (int k = 0; k < 4; k++) {
        DSTimer t;
        for (int r = 0; r < 1000; r++) tests[k](1000);
        printf("%-16s%9.2f ms\n", names[k], t.millis());
    }
}
```

insert at front 91.24 ms

push_back 4.57 ms

with reserve 3.57 ms

direct index 1.50 ms

The timed statement is a call to `test1`, `test2`, `test3`, or `test4`. Because every candidate is compiled in the same program and repeated equally, the comparison exposes the very different amount of element movement.

Each candidate runs 1,000 times on 1,000 elements. Repetition makes the signal large enough to measure; it does not change the asymptotic class.

Each candidate runs 1,000 times on 1,000 elements. Repetition makes the signal large enough to measure; it does not change the asymptotic class.

`insert(begin(), value)` is slow because it shifts the current contents on every insertion. `push_back`, reserved `push_back`, and direct indexed assignment avoid those repeated front shifts.

You can look at the table below to see the Big O efficiency of all the basic vector operations.

You can look at the table below to see the Big O efficiency of all the basic vector operations.

<code>std::vector</code> operation	Complexity
<code>v[i]</code> , assignment to <code>v[i]</code>	$O(1)$
<code>push_back()</code>	amortized $O(1)$; one reallocation can be $O(n)$
<code>pop_back()</code>	$O(1)$
<code>insert(pos, x)</code> , <code>erase(pos)</code>	$O(n)$ in the worst case
<code>reserve(k)</code> when capacity grows	$O(n)$
<code>size()</code> , <code>begin()</code> , <code>end()</code>	$O(1)$
linear search	$O(n)$

The cost of `insert` / `erase` depends on how many elements follow `pos` .

You may be wondering about the two different times for removing an element. When `pop_back()` removes from the end of the vector it takes $O(1)$, but `erase(v.begin())` — removing the first element, or anywhere in the middle — is $O(n)$, because all elements after the removal point must shift left.

You may be wondering about the two different times for removing an element. When `pop_back()` removes from the end of the vector it takes $O(1)$, but `erase(v.begin())` — removing the first element, or anywhere in the middle — is $O(n)$, because all elements after the removal point must shift left.

```
In [13]: #include <iostream>
#include <vector>
#include "pythonds3/cppds/dstimer.hpp"
using namespace std;

int main() {
    // time just the single erase/pop statement, 1000 times each
    vector<int> x(2000000);
    DSTimer t1;
    for (int r = 0; r < 1000; r++) x.erase(x.begin());
    printf("erase(begin()): %10.5f ms\n", t1.millis());

    vector<int> y(2000000);
    DSTimer t2;
    for (int r = 0; r < 1000; r++) y.pop_back();
    printf("pop_back():      %10.5f ms\n", t2.millis());
    return 0;
}
```

erase(begin()): 161.16920 ms

pop_back(): 0.00370 ms

The above shows one attempt to measure the difference between the two removals. Popping from the end is much faster than erasing from the beginning. However, this does not validate the claim that `erase(begin())` is $O(n)$ while `pop_back()` is $O(1)$. To validate that claim we need to look at the performance of both calls over a range of vector sizes:

The above shows one attempt to measure the difference between the two removals. Popping from the end is much faster than erasing from the beginning. However, this does not validate the claim that `erase(begin())` is $O(n)$ while `pop_back()` is $O(1)$. To validate that claim we need to look at the performance of both calls over a range of vector sizes:

```
In [14]: #include <iostream>
#include <vector>
#include "pythonds3/cppds/dstimer.hpp"
using namespace std;

int main() {
    printf("%-10s%14s%12s\n", "n", "erase(begin)", "pop_back");
    for (int n = 2500000; n <= 10000000; n += 2500000) {
        vector<int> x(n);
        DSTimer te;
        for (int r = 0; r < 100; r++) x.erase(x.begin());
        double eraseT = te.millis();
        vector<int> y(n);
        DSTimer tp;
        for (int r = 0; r < 100; r++) y.pop_back();
        printf("%-10d%14.5f%12.5f\n", n, eraseT, tp.millis());
    }
}
```

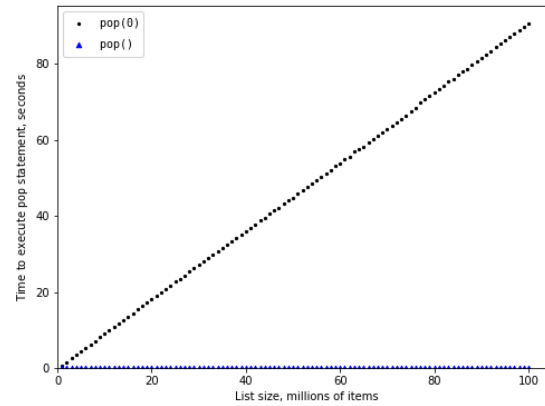
n erase(begin) pop_back

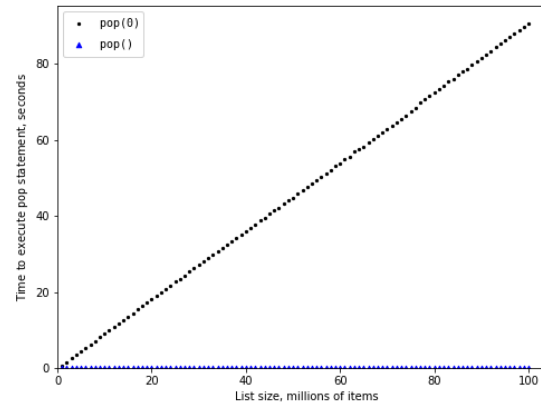
2500000 20.32259 0.00210

5000000 44.32566 0.00023

7500000 82.26057 0.00021

10000000 151.94517 0.00023





As n grows, `erase(begin())` time grows with the number of shifted elements, while `pop_back()` remains approximately flat. Measurements vary by machine, but the trend agrees with the $O(n)$ versus $O(1)$ analysis.

2.7 C++ Strings: `std::string`

Since C++11, `std::string` stores characters contiguously. Its cost model therefore resembles `vector<char>`: indexing and `size()` are $O(1)$, appending is amortized $O(1)$, and insertion or erasure near the front is $O(n)$ because characters shift. Searching for a general substring is not guaranteed to be logarithmic.

Since C++11, `std::string` stores characters contiguously. Its cost model therefore resembles `vector<char>`: indexing and `size()` are $O(1)$, appending is amortized $O(1)$, and insertion or erasure near the front is $O(n)$ because characters shift. Searching for a general substring is not guaranteed to be logarithmic.

<code>std::string</code> operation	Complexity
<code>s[i]</code> , assignment to <code>s[i]</code> , <code>size()</code>	$O(1)$
<code>push_back()</code>	amortized $O(1)$
<code>pop_back()</code>	$O(1)$
<code>insert(pos, text)</code> , <code>erase(pos, count)</code>	$O(n)$ in the worst case
comparison / general substring search	up to $O(n)$ or more, depending on operands/algorithm

As with `vector`, growth policy is implementation-dependent; only the stated complexity guarantees are portable.

```
In [15]: #include <iostream>
#include <string>
using namespace std;

int main() {
    string s = "data";
    s.push_back('!');
    s.insert(0, "C++ ");
    cout << s << " | size=" << s.size() << endl;
}
```

C++ data! | size=9

2.8 Hash Tables: `std::unordered_map`

The second major `C++` associative structure is the **hash table**, `unordered_map`. As you probably recall, hash tables differ from vectors in that you can access items by a **key** rather than a position. Later in this course you will see that there are many ways to implement a hash table!

The second major C++ associative structure is the **hash table**, `unordered_map`. As you probably recall, hash tables differ from vectors in that you can access items by a **key** rather than a position. Later in this course you will see that there are many ways to implement a hash table!

For `std::unordered_map`, lookup, insertion, and erasure are $O(1)$ **on average** when hashing distributes keys well. They are not worst-case constant-time guarantees; a bad collision pattern can make an operation $O(n)$.

Operation	Big O Efficiency (average)
copy	$O(n)$
get item <code>[]</code>	$O(1)$
set item <code>[]</code>	$O(1)$
delete <code>erase</code>	$O(1)$
contains <code>count</code>	$O(1)$
iteration	$O(n)$

Operation	Big O Efficiency (average)
copy	$O(n)$
get item <code>[]</code>	$O(1)$
set item <code>[]</code>	$O(1)$
delete <code>erase</code>	$O(1)$
contains <code>count</code>	$O(1)$
iteration	$O(n)$

These hash-table bounds are **average-case**, not amortized merely because they are averages. Average-case analysis assumes a distribution of hash outcomes; amortized analysis instead spreads occasional expensive work, such as a rehash, over a sequence of operations. A single operation may still be $O(n)$ in the worst case.

For our last performance experiment we will compare the performance of the `contains` operation between vectors and hash tables. In the process we will confirm that searching a vector is $O(n)$ while checking a key in an `unordered_map` is $O(1)$.

For our last performance experiment we will compare the performance of the `contains` operation between vectors and hash tables. In the process we will confirm that searching a vector is $O(n)$ while checking a key in an `unordered_map` is $O(1)$.

```
In [16]: #include <unordered_map>
#include "pythonds3/cppds/dstimer.hpp"
using namespace std;    // iostream/vector/algorithm preloaded by the kernel

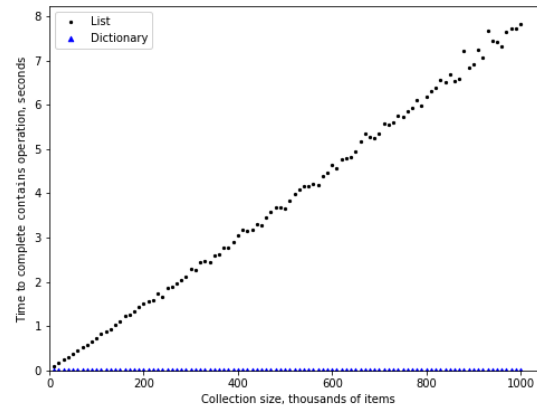
int main() {
    printf("%-10s%10s%12s\n", "n", "vector", "hash table");
    for (int n : {250000, 500000, 1000000}) {
        vector<int> x(n);
        unordered_map<int, int> m;
        for (int j = 0; j < n; j++) { x[j] = j; m[j] = 0; }
        int hits = 0;
        DSTimer tv;
        for (int r = 0; r < 100; r++)
            hits += (find(x.begin(), x.end(), rand() % n) != x.end());
        double vecT = tv.millis();
        DSTimer tm;
        for (int r = 0; r < 100; r++) hits += m.count(rand() % n);
        printf("%-10d%10.3f%12.3f\n", n, vecT, tm.millis());
    }
}
```

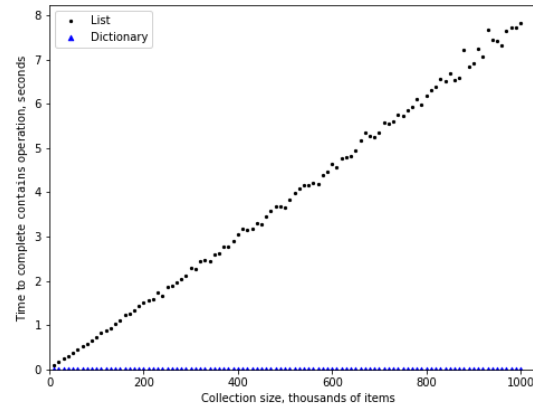
n vector hash table

250000 54.829 0.032

500000 109.228 0.035

1000000 183.135 0.044





The hash table is consistently faster for membership tests in this experiment. Linear search in the `vector` grows with n , whereas `unordered_map::count` stays roughly constant on average. This is empirical support for the analysis, not a worst-case guarantee.

```
In [ ]: display_quiz(path+"dict1.json", max_width=800)
```

Which of the vector operations shown below is NOT $O(1)$?

☐ `vec.push_back(x)`

☐ `vec.erase(vec.begin())`

☐ `vec[10]`

☐ `vec.pop_back()`

In []: `display_quiz(path+"dict2.json", max_width=800)`

Which of the `unordered_map` operations shown below is $O(1)$ on average?

☐ `amap["x"] = amap["x"] + 1`

☐ `amap["x"] == 10`

☐ `amap.count("x")`

☐ All are $O(1)$

☐ `amap.erase("x")`

Exercise: Devise an experiment to verify that `vector::operator[]` is $O(1)$.

Exercise: Devise an experiment to verify that `vector::operator[]` is $O(1)$.

```
In [17]: // Your code here
```

```
In [ ]: from jupytercards import display_flashcards  
display_flashcards("flashcards/ch2.json")
```

Readability



Next



References

1. Textbook: *Problem Solving with Algorithms and Data Structures using C++* (cppds), Chapter 2 —
<https://runestone.academy/ns/books/published/cppds/index.html>

